

姓名: 姜佳

成绩:

03.24 BSP 项目组笔试试题

1. 请写出 printf 的值, 并简单描述 struct 与 union 的区别:

```
main()
{
    union{
        int i;
        struct{
            char first;
            char second;
        }half;
    }number;
    number.i=0x4241;
    printf("%c%c\n", number.half.first, number.half.second);
    number.half.first='a';
    number.half.second='b';
    printf("%x\n", number.i);
    getch();
}
```

答: 1) NULL NULL

2) 0x851

联合体中可以嵌套结构体

3. 假设在 C++ 程序中需要调用一个用 C 编译器编译的函数 `int SubAFunction(x, const char *y)`, 该函数在头文件里应怎样声明? 而在 C 程序中如果也需要调用该函数, 又该如何声明? (加分题, 可以不回答, 请说出这两种声明的原理。)

C 语言中用 `extern` 来调用函数。

原因: 属于外部调用, 加 `extern` 即告诉编译器和链接器。

- 并用于 C 语言。

4. 类 A 派生 B 和 C, 类 D 从 B/C 派生, 如何将一个类 A 的指针指向一个类 D 的实例, 写出代码。

Class ~~String~~ A

{
public:

Class B;

Class C;

Class B

{
public:

Class D;

Class B

Class C

5. 请写出运行结果:

`A // test1`

`union V {`

`struct X {`

`unsigned char s1[3];`

`unsigned char s2[3];`

`unsigned char s3[2];`

`} X;`

`A = B`

`unsigned char c;`

B) // test2

```
(
    char str[] = "world"; const <= sizeof(str) <= 6;
    char *p = str; const <= sizeof(p) <= 4;
    char i = 10; const <= sizeof(i) <= 1;
    void *pp = malloc(10); const <= sizeof(pp) <= 4;
)
```

6 : 4 : 1 : 4

6. 请简述进程与线程的关系，并列举进（线）程间同步机制有哪些，线程间有哪些手段通信通信？（linux 或 windows 都可以）

linux 下：进程间通信方式主要有三类

1. 管道 } Pipe
2. 信号量 } Semaphore
3. 消息队列 } Message Queue

线程间通信：线程由进程创建，多个线程属于一个进程下的资源，线程对系统资源开销小，但线程的创建和销毁开销较大。

7. 为什么 virtual 在 C++ 中的基类里常做虚函数？

2-CP 或多 CP 间的通信。

8. new/delete 与 malloc/free 的联系与区别？

区别：malloc/free 属于 C 语言中，new/delete 属于 C++ 语言中。

联系：都是用于申请、释放内存空间，但 new/delete 是 C++ 编译器自动调用的，而 malloc/free 是程序员手动调用的。

9. 一个链表的结点结构如下：

```
struct Node
```

```
{
```

```
int data;
```

```
Node *next;
```

```
typedef struct Node node;
```


10. 头文件中的 #ifndef/#define/#endif 作用是什么?

答: 为了防止某个代码段被重复定义, 主要是因为安全起见, 防止重复定义。若之前未被定义, 则在它上面便进行定义; 若之前已定义过, 则不再重复定义。

11. #include <filename.h> 和 #include "filename.h" 有什么区别?

答: 前者编译器会从它自己指定的系统路径里去寻找 filename.h; 后者编译器会在当前路径下寻找文件 filename.h。

12. 关键字 static 的作用是什么?

加上 static 后就 ~~变成~~ 是使得后面的字段定义为静态的, 或者说是在局部的, 即只能在当前文件中被调用。

13. 描述关键字 const 是程序设计中有那些作用(至少 2 点), 以及下面变量定义的含义是什么?

const int *a;

int * const a;

答: 加 const 即声明为“只读的”。

防止被修改。

主要是为了安全起见。

从 const int *a: 定义了一个常量指针。

从 int * const a: 定义了一个指针常量。

好处: ~~变量在编译时就被定义为只读的, 因此不能被修改。~~

可以在编译及编译阶段就被修改。

14. 2019-2020 年 12 月 31 日止，本公司（含子公司）
 之 2019 年度综合收益总额，按 2019 年度综合收益总额
 的 10% 提取法定盈余公积，按 5% 提取任意盈余公积，
 按 5% 提取一般风险准备，按 5% 提取其他综合收益。
 2019 年度综合收益总额 100,000,000.00 元

15. 2019 年度综合收益总额，按 2019 年度综合收益总额的 10% 提取法定盈余公积，按 5% 提取任意盈余公积，按 5% 提取一般风险准备，按 5% 提取其他综合收益。

2019 年度综合收益总额 100,000,000.00 元

16. 2019 年度综合收益总额，按 2019 年度综合收益总额的 10% 提取法定盈余公积，按 5% 提取任意盈余公积，按 5% 提取一般风险准备，按 5% 提取其他综合收益。

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元
 2019 年度综合收益总额 100,000,000.00 元
 2019 年度综合收益总额 100,000,000.00 元
 2019 年度综合收益总额 100,000,000.00 元
 2019 年度综合收益总额 100,000,000.00 元



17. 2019 年度综合收益总额，按 2019 年度综合收益总额的 10% 提取法定盈余公积，按 5% 提取任意盈余公积，按 5% 提取一般风险准备，按 5% 提取其他综合收益。

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元

2019 年度综合收益总额 100,000,000.00 元

18. 结合 16 和 17 题，思考说明 heap 和 stack 区别？

答：heap：申请时一直占用内存有空间。

stack：从最高地址开始向低地址内存，程序运行结束时释放内存空间。

19. 下面是一个基于 ARM 架构（或者其它架构）的 C 程序，请指出错误（unassigned int dma_address）

```
{
    int status = 0;
    while(status = *(unassigned int *dma_address) == 0, dma_address 是 DMA 寄存器, dma) = 1; 表明传输完毕
    return 1;
}
```

20. 请查找下面中断服务子程序(ISR)，有哪些错误？

```
double compute_area (double radius) interrupt 0 using 2
{
    double area = PI * radius * radius;
    printf("Area %f", area);
    return area;
}
```


14. 2019-2020 年 12 月 31 日止，本公司（含子公司）
 之 2019 年度综合收益总额，按 2019 年度综合收益总额
 的 10% 提取法定盈余公积，按 5% 提取任意盈余公积，
 按 5% 提取一般风险准备，按 5% 提取其他综合收益。
 2019 年度综合收益总额 100,000,000.00 元

15. 2019 年度综合收益总额，按 2019 年度综合收益总额的 10% 提取法定盈余公积，按 5% 提取任意盈余公积，按 5% 提取一般风险准备，按 5% 提取其他综合收益。

2019 年度综合收益总额 100,000,000.00 元
 按 10% 提取法定盈余公积 10,000,000.00 元
 按 5% 提取任意盈余公积 5,000,000.00 元
 按 5% 提取一般风险准备 5,000,000.00 元
 按 5% 提取其他综合收益 5,000,000.00 元
 2019 年度综合收益总额 100,000,000.00 元
 按 10% 提取法定盈余公积 10,000,000.00 元
 按 5% 提取任意盈余公积 5,000,000.00 元
 按 5% 提取一般风险准备 5,000,000.00 元
 按 5% 提取其他综合收益 5,000,000.00 元



17. 2019 年度综合收益总额，按 2019 年度综合收益总额的 10% 提取法定盈余公积，按 5% 提取任意盈余公积，按 5% 提取一般风险准备，按 5% 提取其他综合收益。

2019 年度综合收益总额 100,000,000.00 元
 按 10% 提取法定盈余公积 10,000,000.00 元
 按 5% 提取任意盈余公积 5,000,000.00 元
 按 5% 提取一般风险准备 5,000,000.00 元
 按 5% 提取其他综合收益 5,000,000.00 元